
nobody Design Outline

Release 0.1 draft

nobody contributors

May 11, 2026

CONTENTS

1	Introduction	3
1.1	Ambient Authority Problem	3
1.2	Design Goals	3
1.3	Current Prototype	4
1.4	Guide to Reading This Document	4
2	High-Level Abstractions	5
2.1	Actors, Subjects, and Runs	5
2.2	Capabilities	5
2.3	Policies	5
2.4	Decisions	5
2.5	Traces	5
3	Conceptual Design	7
3.1	Authority Model	7
3.2	Capability Graph	7
3.3	Run Lifecycle	7
3.4	Enforcement Boundaries	8
3.5	Approval Gates	8
3.6	Evidence Model	8
4	Technical Design	9
4.1	Workspace Layout	9
4.2	Command-Line Interface	9
4.3	Policy Parsing and Validation	10
4.4	Runtime Supervision	10
4.5	Environment Filtering	10
4.6	Process Enforcement	10
4.7	Filesystem Enforcement	10
4.8	Network Enforcement	10
4.9	MCP and Tool Mediation	10
4.10	Secret Handling	11
4.11	Trace Writing	11
4.12	Replay and Diff	11
4.13	Error Handling	11
4.14	Configuration	11
4.15	Platform Support	11
A	Glossary	13
B	Policy Schema	15

B.1	Agent and Task	15
B.2	Filesystem and Network	15
B.3	Process and Environment	15
B.4	Approvals and Trace	15
C	Threat Model	17
D	Deployment	19
E	Roadmap	21

Preface

nobody is a least-privilege execution runtime for autonomous software. It runs agents, tools, MCP servers, and shell commands with declared capabilities instead of ambient authority inherited from the launching shell.

This document gives a brief introduction to what nobody aims to achieve and the design elements in support of those aims.

INTRODUCTION

nobody treats each run as an authority-bearing object. A run starts with an actor, a subject, and a policy. The runtime evaluates the policy, applies the available enforcement mechanisms, starts the subject only when the boundary is valid, and records the resulting events as trace evidence.

The current prototype is intentionally small. It parses typed policy from `nobody.toml`, evaluates process and environment rules, filters inherited environment variables, applies Linux Landlock filesystem boundaries when available, runs allowed commands, and writes JSONL trace events.

1.1 Ambient Authority Problem

A developer shell commonly has broad access to source trees, credentials, package registries, local sockets, browsers, and internal networks. A normal child process inherits too much of that authority by default.

Autonomous software makes that failure mode sharper. An agent can inspect, plan, retry, call tools, and combine local and remote actions. The design goal is therefore not just to log what happened, but to reduce what the subject can do in the first place.

1.2 Design Goals

The runtime should:

- make authority explicit in a reviewable policy file,
- evaluate requests as allow, deny, or ask decisions,
- enforce the strongest boundary available for each capability class,
- record enough evidence to explain every material decision, and
- keep the main interface small enough to compose with normal Unix tools.

1.3 Current Prototype

The current repository enforces process allow/deny decisions before spawn and filters inherited environment variables before the child process starts. On Linux it also enforces filesystem read/write grants through Landlock when the policy does not require deny carve-outs beneath granted paths. Network, MCP, browser, secret, and approval boundaries are parsed as design-facing policy sections but are not yet enforced by the runtime.

1.4 Guide to Reading This Document

This outline separates the design into four chapters. The introduction states the problem and current implementation status. The high-level abstractions define the vocabulary used by the design. The conceptual design describes the authority model independent of implementation. The technical design maps that model onto the Rust workspace and future enforcement mechanisms.

HIGH-LEVEL ABSTRACTIONS

The design uses a small vocabulary so that policy, runtime behavior, and trace evidence can be inspected together.

2.1 Actors, Subjects, and Runs

An *actor* is the human, CI job, service account, or organization principal that starts work. A *subject* is the command, process tree, agent, MCP server, or tool being executed. A *run* is the bounded execution that joins the actor, subject, policy, confinement plan, trace, and final result.

2.2 Capabilities

A *capability* is a typed grant of authority over a resource or operation. A grant to read a path is not a grant to write it. A grant to call one MCP tool is not a grant to call another. A grant to use a secret is scoped to an approved sink.

2.3 Policies

A policy is the source of declared authority for a run. The first policy format is TOML because it is easy to review, diff, and keep in a repository. The runtime should reject invalid policy rather than widening authority.

2.4 Decisions

Policy evaluation returns a decision and a reason. The decision is one of `allow`, `deny`, or `ask`. The reason identifies the matched rule, matched pattern, target resource, and operation.

2.5 Traces

A trace is append-only evidence for a run. It records what was requested, which decision was made, why that decision was made, and what effect followed.

CONCEPTUAL DESIGN

The conceptual design separates authority, enforcement, and evidence. Policy describes what a subject may do. Enforcement mechanisms make those decisions true where possible. Trace events make the decisions and observed effects auditable after the run.

3.1 Authority Model

The intended authority relation is a total function over policy, request, and context:

```
eval(policy, request, context) -> decision, reason
```

The reason is part of the contract. A boundary that cannot explain why a request was allowed or denied cannot be audited by inspection.

3.2 Capability Graph

Capabilities form a graph from actor to subject to resources. The graph should make delegation explicit: the actor has broad authority, but the subject receives only the subset named in policy for the current run.

3.3 Run Lifecycle

The run lifecycle is:

1. load policy,
2. validate and normalize policy,
3. construct an enforcement plan,
4. create trace state,
5. evaluate the subject launch request,
6. start or deny the subject,
7. record runtime events, and
8. finalize the trace summary.

3.4 Enforcement Boundaries

Different capabilities require different mechanisms. Filesystem operations may use Landlock, mount namespaces, overlays, and path policy. Process execution uses allow/deny checks, namespace inheritance, seccomp, and resource limits. Network access uses a deny-default network namespace, DNS policy, and an egress proxy. MCP tools and secrets require mandatory user-space mediation.

3.5 Approval Gates

An approval gate represents a request that policy cannot automatically allow. The runtime should suspend the request, identify the approver, record the decision, and continue only with the approved authority.

3.6 Evidence Model

Evidence is not diagnostic noise. It is the record used to inspect a run, answer why an action happened, reconstruct authority use, and export audit material.

TECHNICAL DESIGN

The technical design maps the authority model onto a Rust workspace. Dependency direction matters: policy types should not depend on Linux confinement, and trace readers should not depend on the command-line interface.

4.1 Workspace Layout

The current workspace is split into focused source directories:

crates/cli

Command parsing and terminal-facing behavior.

crates/policy

Policy structures, validation, and decision evaluation.

crates/runtime

Run lifecycle, process supervision, and environment filtering.

crates/sandbox

Platform sandbox interfaces plus noop and Linux Landlock backends.

crates/trace

Trace event schema, JSONL writing, and compact trace display.

4.2 Command-Line Interface

The primary interface remains:

```
nobody run --policy nobody.toml -- <command>
```

Supporting commands inspect policy and trace output without broadening the authority granted to the subject.

4.3 Policy Parsing and Validation

The parser accepts agent, task, filesystem, network, process, environment, approval, and trace sections. The prototype enforces process, environment, and Linux filesystem sections first while preserving the larger file shape for future enforcement.

4.4 Runtime Supervision

The runtime owns run creation, preflight checks, child process launch, exit status collection, and trace finalization.

4.5 Environment Filtering

Environment filtering prevents the subject from inheriting credentials and machine state simply because the actor had them. Trace events record variable names and counts, not values.

4.6 Process Enforcement

Process policy is enforced before spawn. Denied commands do not run, and the denial is recorded in the trace.

4.7 Filesystem Enforcement

The first filesystem backend uses Linux Landlock. The runtime prepares a sandbox specification from `fs.read`, `fs.write`, and `fs.deny` and installs fully enforced ABI v3 filesystem rules in the child process after fork and before exec. Non-Linux hosts use a noop backend and emit a warning.

Landlock grants access to path trees; it cannot subtract a denied child from an already-granted parent. The runtime therefore fails closed for policies such as `read = ["."]` with `deny = [".env"]` on Linux instead of silently oversteating enforcement.

4.8 Network Enforcement

Network policy should default to deny and route permitted egress through a mediated path that records destination decisions. The prototype does not yet enforce network access.

4.9 MCP and Tool Mediation

Tool calls should be treated as capability-bearing operations. A future MCP proxy will inspect tool schemas, evaluate requests, and enforce allow, deny, or approval decisions.

4.10 Secret Handling

Secrets should be brokered into approved sinks rather than inherited through the process environment. Trace output must avoid recording secret values.

4.11 Trace Writing

Trace output is newline-delimited JSON. Each event carries schema version, run identifier, event identifier, timestamp, actor, event kind, optional decision, and payload data.

4.12 Replay and Diff

Replay initially means reconstruction of authority history, not deterministic re-execution of arbitrary programs. Diffing should compare what authority was requested, granted, denied, and observed across runs.

4.13 Error Handling

Errors in policy validation, unavailable enforcement mechanisms, failed proxies, or failed secret brokers must fail closed. Failure should not widen authority.

4.14 Configuration

The default policy path is `nobody.toml` and the default trace path is `.nobody/runs/latest.jsonl`. Both should remain visible and easy to inspect.

4.15 Platform Support

The first strong enforcement target is Linux. Other platforms may support policy evaluation and trace generation before they support equivalent kernel boundaries.

GLOSSARY

actor

The human, CI job, service account, or organization principal that starts a run.

subject

The command, process tree, agent, MCP server, or tool being executed.

capability

A typed grant of authority over a resource or operation.

policy

The reviewable declaration of authority for a run.

decision

The result of evaluating a request against policy: allow, deny, or ask.

trace

Append-only event evidence describing a run.

POLICY SCHEMA

The policy schema is intentionally plain. TOML keeps the first design reviewable in normal code review and easy to generate from other systems later.

B.1 Agent and Task

The agent section names the subject class. The task section describes the work context, such as a repository or task identifier.

B.2 Filesystem and Network

Filesystem policy declares readable, writable, and denied paths. Network policy declares the default mode and endpoint allow/deny lists.

Linux filesystem enforcement is based on grantable path trees. Deny entries are useful for policy explanation, but a denied path beneath an allowed parent cannot be represented by Landlock and must fail closed at runtime.

B.3 Process and Environment

Process policy declares allowed and denied programs. Environment policy declares whether inherited variables are cleared and which variable names may be kept.

B.4 Approvals and Trace

Approval policy names operations that must suspend for explicit approval. Trace policy selects the event output path and redaction patterns.

THREAT MODEL

The design reduces the authority of a subject under a stated policy. It is not a proof that an arbitrary host is safe.

In scope:

- a subject attempting to run denied commands,
- a subject attempting to read or write denied paths,
- arbitrary network egress,
- unauthorized MCP tool calls,
- secret use outside approved sinks, and
- irreversible actions that require approval.

Out of scope initially:

- a compromised host kernel,
- a malicious root administrator,
- hardware side channels,
- perfect data-flow tracking through arbitrary process memory, and
- full deterministic replay of all programs.

DEPLOYMENT

The core runtime is a local binary. A hosted service can distribute policy, store approvals, summarize traces, integrate identity, and export evidence, but the boundary should remain enforceable without a hosted control plane.

The first deployment targets are developer workstations and CI runners. Managed runners can use the same policy and trace model once the local primitive is stable.

ROADMAP

The implementation should move from semantics to enforcement:

1. 0.3: Linux filesystem enforcement,
2. 0.4: escape-test suite,
3. 0.5: process and environment hardening,
4. 0.6: trace viewer and policy diagnostics polish,
5. 0.7: network egress enforcement,
6. 0.8: agent profiles,
7. 0.9: MCP proxy, and
8. 1.0: stable local agent runtime.

The immediate post-Landlock priority is the escape suite, not network. The tests should prove that allowed interpreters and build systems remain unable to read denied files.